ICS 433 – Operating Systems

Distributed Resource Scheduler

Phase 2 Progress Report

King Fahd University of Petroleum & Minerals

Rayan Alamri Mohammed Alsuhaibani Ali Almuzain Mohammed Yar

April 21, 2026

Contents

1 Project Recap **3**
1.1 Overview . . . 3
1.2 Motivation . . . 3
1.3 Team Responsibilities . . . 3

2 What Has Been Implemented **3**
2.1 Master Node (Rayan Alamri) — 100% Complete . . . 3
2.1.1 TCP Server (`src/master/server.c`) . . . 3
2.1.2 Worker Registry (`src/master/scheduler.c`) . . . 4
2.1.3 Task Queue (`src/master/queue_mgr.c`) . . . 4
2.1.4 Scheduler Thread (`src/master/scheduler.c`) . . . 4
2.2 Worker Execution Engine (Mohammed Alsuhaibani) — 100% Complete . 4
2.2.1 Network Client (`src/worker/client.c`) . . . 4
2.2.2 Executor (`src/worker/executor.c`) . . . 5
2.3 Serialization & Communication Layer (Ali Almuzain) — 100% Complete 5
2.3.1 Protocol (`src/shared/protocol.c`, `src/shared/models.h`) . . . 5
2.4 Terminal Dashboard (Mohammed Yar) — Stub Only . . . 5
2.5 Docker / Deployment Infrastructure . . . 5

3 System Architecture **6**
3.1 Component Diagram . . . 6
3.2 Message Flow . . . 6
3.3 Concurrency Model . . . 7

4 Screenshots of Working System **7**
4.1 Master Node Startup and Worker Registration . . . 7
4.2 Task Dispatch and Result Collection . . . 7
4.3 Worker Node Output . . . 8
4.4 Unit Test Output . . . 8
4.5 Fault Tolerance — Worker Disconnect and Task Requeue . . . 8

5 How to Run the System **8**
5.1 Build . . . 9
5.2 Run Unit Tests . . . 9
5.3 Run Integration Test (Local — 3 Workers) . . . 9
5.4 Run with Docker . . . 9
5.5 Test Fault Tolerance . . . 9

6 Challenges Faced **10**
6.1 Thread Safety Without Deadlock . . . 10
6.2 TCP Partial Reads . . . 10
6.3 Worker Disconnect Race . . . 10
6.4 Child Process Cleanup . . . 10
6.5 Heartbeat and Send Interleaving . . . 10

7 Implementation Progress Summary **11**

8 Plan for Completing Remaining Work **11**
8.1 Remote Code Execution on Workers (Core Priority) . . . 11
8.2 Terminal Dashboard (Mohammed Yar) . . . 11
8.3 Heartbeat Watchdog . . . 12
8.4 Result Logging . . . 12
8.5 Additional Test Coverage . . . 12
8.6 Raspberry Pi Deployment (Time-Permitting) . . . 12

9 Repository Structure **13**

1 Project Recap

1.1 Overview

The Distributed Resource Scheduler is a **Pure C** system that pools underutilized computing power across a Local Area Network (LAN) using an asynchronous **Master–Worker** architecture. Its goal is to coordinate multiple worker nodes and assign computational tasks efficiently, demonstrating core OS concepts including multithreading, inter-process communication (IPC), process isolation, and network socket programming — all without any external runtime libraries beyond POSIX and `pthreads`.

1.2 Motivation

Modern distributed systems (e.g., Hadoop, Kubernetes) schedule work across heterogeneous machines. This project implements a minimal but functionally correct version of such a system in pure C, targeting POSIX-compliant Unix environments (Linux, macOS, WSL). The constraints — no frameworks, no scripting languages — force direct engagement with system calls: `fork()`, `pipe()`, `socket()`, `pthread_create()`, `send()`, and `recv()`.

1.3 Team Responsibilities

Team Member	Module
Rayan Alamri	Master Node (server, scheduler, queue, registry)
Mohammed Alsuhaibani	Worker Execution Engine (client, executor)
Ali Almuzain	Serialization & Communication Layer (protocol)
Mohammed Yar	Terminal Dashboard (ncurses UI)

2 What Has Been Implemented

As of Phase 2, approximately **75–80%** of the system is complete. The networking core, scheduling engine, worker execution pipeline, and serialization layer are all fully implemented and tested. The terminal dashboard remains as a stub.

2.1 Master Node (Rayan Alamri) — 100% Complete

2.1.1 TCP Server (`src/master/server.c`)

- Binds a `SOCK_STREAM` socket on a configurable port (default 9090, overridable via `MASTER_PORT` environment variable).
- Runs an infinite `accept()` loop to handle new worker connections.
- Validates an initial `MSG_REGISTER` handshake before admitting a worker into the registry, preventing random TCP clients from polluting scheduler state.
- Spawns a **detached** `pthread` per connected worker to receive results and heartbeats concurrently.
- On worker disconnection: detects EOF via `recv_full()`, requeues any in-flight task, and marks the worker offline.

- Includes a demo task seeder thread (activated via `DEMO_TASKS` environment variable) that enqueues configurable sample tasks 3 seconds after startup.

2.1.2 Worker Registry (`src/master/scheduler.c`)

- Fixed-size array of `WorkerInfo` structs (up to 32 workers, defined by `MAX_WORKERS`).
- Each entry tracks: `worker_id`, `sock_fd`, `status` (`IDLE`/`BUSY`/`OFFLINE`), `last_heartbeat` timestamp, and a snapshot of the current task.
- All mutations protected by `pthread_mutex_lock()` / `pthread_mutex_unlock()`.
- Operations: `registry_add()`, `registry_set_status()`, `registry_set_offline()`, `registry_find_idle()`, `registry_get()`.
- Slot reuse: offline workers free their array index so it can be claimed by new registrations without compacting the table (keeping pointers stable for concurrent readers).

2.1.3 Task Queue (`src/master/queue_mgr.c`)

- Dynamically allocated **FIFO linked list** of `NetworkPayload` nodes.
- Thread-safe via `pthread_mutex_t` + `pthread_cond_t not_empty`.
- Operations: `queue_enqueue()`, `queue_dequeue()` (blocking), `queue_dequeue_nowait()` (non-blocking), `queue_requeue()` (re-inserts at tail), `queue_size()`, `queue_is_empty()`, `queue_destroy()`.
- `queue_destroy()` drains all nodes to prevent memory leaks between test runs.

2.1.4 Scheduler Thread (`src/master/scheduler.c`)

- Runs as a background `pthread`.
- Implements **Round-Robin** scheduling via a persistent `rr_index` cursor over the worker registry.
- Uses `queue_dequeue_nowait()` in a polling loop (5 ms sleep when idle, 10 ms when no workers are free) to avoid blocking the shutdown path.
- Marks a worker `BUSY` before sending the task; on send failure, requeues the task and marks the worker offline.
- Lifecycle: `scheduler_init()`, `scheduler_start()`, `scheduler_stop()` (sets `running=0` and `pthread_joins` the thread).

2.2 Worker Execution Engine (Mohammed Alsuhaibani) — 100% Complete

2.2.1 Network Client (`src/worker/client.c`)

- Connects to the master using `getaddrinfo()` for DNS resolution and `IPv4`/`IPv6` agnosticism.
- Accepts host/port/worker-ID via CLI arguments or environment variables (`MASTER_HOST`, `MASTER_PORT`, `WORKER_ID`).
- Sends a `MSG_REGISTER` handshake immediately after connection.
- Spawns a **heartbeat thread** that sends `MSG_HEARTBEAT` packets every 5 seconds; on send failure, shuts down the socket to unblock the main receive loop.
- Per-task execution is dispatched to a new `pthread` so the receive loop remains responsive for concurrent control messages.

- Thread-safe sends via a `send_lock` mutex (shared between heartbeat and task-result threads).
- Graceful shutdown: waits for all in-flight task threads via a condition variable before closing the socket.

2.2.2 Executor (`src/worker/executor.c`)

- `executor_run_task()` forks a child process for each task, providing CPU isolation.
- Uses `pipe()` for parent–child IPC: the child writes its `uint32_t` result and exits; the parent reads it.
- Child uses `_exit()` to prevent flushing parent-owned `stdio` buffers.
- Parent calls `waitpid()` after reading the result to reap the child and check its exit status.
- Two built-in workloads:
  - **Command 1 – Prime Counter:** counts primes up to `argument` using trial division (avoids `sqrt()` via `d <= n/d` to prevent overflow).
  - **Command 2 – Matrix Checksum:** simulates nested-loop CPU work on a virtual matrix of size `argument`×`argument` and returns an accumulated checksum.
- Error codes: `EXECUTOR_OK`, `EXECUTOR_ERR_PIPE`, `EXECUTOR_ERR_FORK`, `EXECUTOR_ERR_CHILD_IO`, `EXECUTOR_ERR_CHILD_STATUS`.

2.3 Serialization & Communication Layer (Ali Almuzain) — 100% Complete

2.3.1 Protocol (`src/shared/protocol.c`, `src/shared/models.h`)

- **Fixed-size payload:** all messages use a single `NetworkPayload` struct (6 × `uint32_t` = 24 bytes), eliminating stream fragmentation and variable-length parsing.
- **Byte-order normalization:** `payload_to_net()` converts all fields to network byte order before sending; `payload_to_host()` converts back on receipt — ensuring cross-architecture compatibility.
- **Reliable I/O:** `recv_full()` and `send_full()` loop until exactly `sizeof(NetworkPayload)` bytes have been transferred, handling TCP's byte-stream nature.
- **Message types:** `MSG_HEARTBEAT` (0), `MSG_TASK` (1), `MSG_RESULT` (2), `MSG_REGISTER` (3).
- **Worker states:** `WORKER_IDLE` (0), `WORKER_BUSY` (1), `WORKER_OFFLINE` (2).

2.4 Terminal Dashboard (Mohammed Yar) — Stub Only

The dashboard module (`src/ui/dashboard.c`) currently contains empty stub implementations for `dashboard_init()`, `dashboard_run()`, and `dashboard_stop()`. The ncurses-based live monitoring UI is the primary remaining deliverable for Phase 3.

2.5 Docker / Deployment Infrastructure

The following deployment tooling has been added:

- **Dockerfile** — multi-stage image for master and worker binaries.

- **docker-compose.yml** — defines master + worker services with resource limits simulating Raspberry Pi 3/4/5 hardware profiles.
- **scale_workers.sh** — launches configurable counts of RPi3/4/5-profile worker containers.
- **manage_workers.sh** — add/remove individual containers at runtime to simulate dynamic node join/leave.
- **start-workers.sh** — convenience script to start N local workers against a given host/port.

3 System Architecture

3.1 Component Diagram

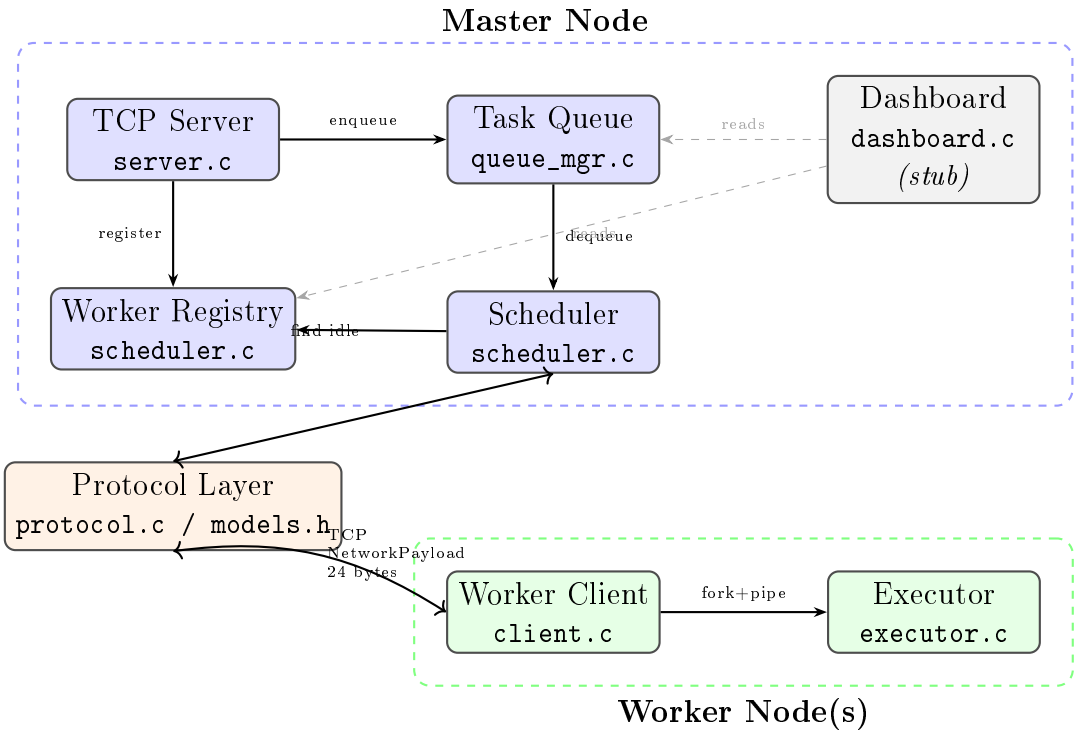

Figure 1: System architecture: Master Node, Worker Node(s), and Protocol Layer.

3.2 Message Flow

1. Worker starts → connects to master via TCP → sends `MSG_REGISTER`.
2. Master assigns a `worker_id`, adds worker to registry as `IDLE`.
3. Scheduler dequeues a task, calls `registry_find_idle()` (round-robin), marks worker `BUSY`, sends `MSG_TASK`.
4. Worker receives `MSG_TASK` → `dispatch_task()` spawns a task thread → `executor_run_task()` forks a child process.
5. Child executes workload → writes result to pipe → exits.

6. Parent reads result from pipe → `wait()`s for child → sends `MSG_RESULT` to master.
7. Master marks worker `IDLE`; cycle repeats.
8. Heartbeats flow every 5 seconds from worker → master; master updates `last_heartbeat`.
9. On disconnect: master requeues any `BUSY` worker's in-flight task.

3.3 Concurrency Model

Process	Thread / Process	Responsibility
Master	Main thread	`accept()` loop
	Per-worker thread	Receive results & heartbeats
	Scheduler thread	Dequeue tasks, dispatch round-robin
	Demo seeder thread	Inject sample tasks (optional)
Worker	Main thread	Receive tasks from master
	Heartbeat thread	Send `MSG_HEARTBEAT` every 5 s
	Per-task thread	Call `executor_run_task()`
	Child process	Execute actual workload (forked)

4 Screenshots of Working System

4.1 Master Node Startup and Worker Registration

Figure 2: Master node listening and workers registering.

4.2 Task Dispatch and Result Collection

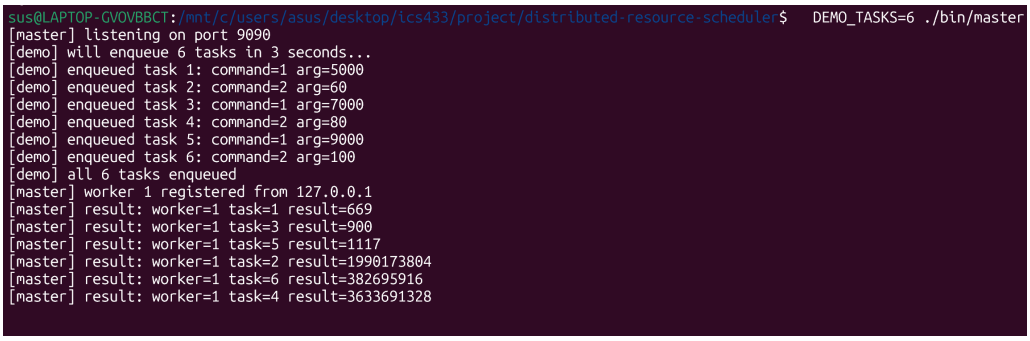

Figure 3: Master dispatching tasks and collecting results from workers.

4.3 Worker Node Output

```
[worker] master disconnected
sus@LAPTOP-GVOVBBCT:/mnt/c/users/asus/desktop/ics433/project/distributed-resource-scheduler$ ./bin/worker 127.0.0.1 9090
[worker] connected to master at 127.0.0.1:9090
[worker] received task=1 command=1 argument=5000
[worker] sent result: task=1 result=669
[worker] received task=3 command=1 argument=7000
[worker] sent result: task=3 result=900
[worker] received task=5 command=1 argument=9000
[worker] sent result: task=5 result=1117
[worker] received task=2 command=2 argument=60
[worker] sent result: task=2 result=1990173804
[worker] received task=6 command=2 argument=100
[worker] sent result: task=6 result=382695916
[worker] received task=4 command=2 argument=80
[worker] sent result: task=4 result=3633691328
```

Figure 4: Worker receiving a task, executing it, and sending the result back.

4.4 Unit Test Output

```
sus@LAPTOP-GVOVBBCT:/mnt/c/users/asus/desktop/ics433/project/distributed-resource-scheduler$ make test
gcc -Wall -Wextra -g -pthread -o bin/test_scheduler tests/test_scheduler.c src/master/queue_mgr.c src/master/scheduler.c
 src/shared/protocol.c
./bin/test_scheduler
=== Master unit tests ===
[PASS] queue_fifo
[PASS] queue_empty
[PASS] queue_requeue
[PASS] registry_add_and_idle
[PASS] registry_round_robin
[PASS] registry_set_offline
[PASS] registry_busy_not_selected
[PASS] payload_byte_order_roundtrip
All tests passed.
gcc -Wall -Wextra -g -pthread -o bin/test_worker_executor tests/test_worker_executor.c src/worker/executor.c src/shared/
protocol.c
./bin/test_worker_executor
=== Worker executor tests ===
[PASS] executor_prime_task
[PASS] executor_matrix_task
[PASS] protocol_socketpair_roundtrip
All worker tests passed.
```

Figure 5: All unit tests passing (`make test`).

4.5 Fault Tolerance — Worker Disconnect and Task Requeue

```
sus@LAPTOP-GVOVBBCT:/mnt/c/users/asus/desktop/ics433/project/distributed-resource-scheduler$ DEMO_TASKS=7 ./bin/master
[master] listening on port 9090
[demo] will enqueue 7 tasks in 3 seconds...
[demo] enqueued task 1: command=1 arg=5000000
[demo] enqueued task 2: command=1 arg=6000000
[demo] enqueued task 3: command=1 arg=7000000
[demo] enqueued task 4: command=1 arg=8000000
[demo] enqueued task 5: command=1 arg=9000000
[demo] enqueued task 6: command=1 arg=10000000
[demo] enqueued task 7: command=1 arg=11000000
[demo] all 7 tasks enqueued
[master] worker 1 registered from 127.0.0.1
[master] worker 2 registered from 127.0.0.1
[master] result: worker=2 task=2 result=412849
[master] result: worker=1 task=6 result=664579
[master] worker 2 disconnected
[master] requeueing orphaned task 4
[master] result: worker=1 task=7 result=726517
```

Figure 6: Master detecting worker disconnect and requeueing the orphaned task.

5 How to Run the System

This section provides the exact commands to run for generating the screenshots above and for verifying the system works end-to-end.

5.1 Build

```
# From the project root:
make all
# Produces: bin/master and bin/worker
```

5.2 Run Unit Tests

```
make test
# Runs both test suites and prints PASS/FAIL for each case.
# Expected output: "All tests passed."
```

To run each suite separately:

```
make test_scheduler      # 8 master unit tests (queue, registry,
   protocol)
make test_worker         # 3 worker unit tests (prime, matrix,
   socket roundtrip)
```

5.3 Run Integration Test (Local — 3 Workers)

Open **four terminals**:

Terminal 1 — Start the master with 6 demo tasks:

```
DEMO_TASKS=6 ./bin/master
```

Terminals 2, 3, 4 — Start workers:

```
./bin/worker 127.0.0.1 9090
```

Or use the provided script to start 3 workers at once:

```
./start-workers.sh 3 127.0.0.1 9090
```

5.4 Run with Docker

```
docker-compose build
./scale_workers.sh 2 1 1    # 2 RPi3 + 1 RPi4 + 1 RPi5 workers
docker-compose logs -f master
```

5.5 Test Fault Tolerance

```
# 1. Start master with many tasks:
DEMO_TASKS=20 ./bin/master

# 2. Start 3 workers:
./start-workers.sh 3 127.0.0.1 9090
```

```
# 3. Wait a few seconds, then kill one worker (Ctrl-C in its
   terminal).
#    The master terminal should print:
#      [master] worker N disconnected
#      [master] requeueing orphaned task X
#    And the surviving workers should complete all tasks.
```

6 Challenges Faced

6.1 Thread Safety Without Deadlock

The master holds two independent mutexes: `registry.lock` and `queue.lock`. Early versions acquired both simultaneously (from within the scheduler thread), which risked deadlock when the server thread also needed the registry lock. The solution was to release `registry.lock` before calling `queue_requeue()`, ensuring the two subsystem locks are never held concurrently.

6.2 TCP Partial Reads

A naive single `recv()` call for a `NetworkPayload` would silently return fewer than 24 bytes under load. Both the master and worker now use `recv_full()` and `send_full()` — looping until exactly `sizeof(NetworkPayload)` bytes have been transferred — which eliminated intermittent message corruption.

6.3 Worker Disconnect Race

When a worker disconnects, its in-flight task must be requeued exactly once. The initial implementation could race: the server thread could set the worker offline before the task snapshot had been captured, losing the task. The fix was to snapshot `current_task` while still holding `registry.lock`, release the lock, *then* call `queue_requeue()`, with a guard on `task_id != 0` to skip workers that were idle at disconnect time.

6.4 Child Process Cleanup

The executor originally leaked zombie processes because the parent `worker_run` loop did not call `waitpid()`. After moving execution into its own thread and calling `waitpid()` inside `executor_run_task()`, child reaping is now deterministic. Using `_exit()` in the child also prevents double-flushing of parent `stdio` buffers.

6.5 Heartbeat and Send Interleaving

The heartbeat thread and task-result thread both write to the same socket. Without synchronization, their bytes would interleave, corrupting the master's protocol parser. A dedicated `send_lock` mutex serializes all socket writes from the worker, regardless of which thread initiates them.

7 Implementation Progress Summary

Module	Status	% Done	Notes
TCP Server (`server.c`)	Complete	100%	Registration, dispatch, disconnect
Task Queue (`queue_mgr.c`)	Complete	100%	FIFO, requeue, thread-safe
Scheduler (`scheduler.c`)	Complete	100%	Round-robin, fault-safe
Worker Registry (`scheduler.c`)	Complete	100%	Slot reuse, mutex-protected
Worker Client (`client.c`)	Complete	100%	Heartbeat, multi-task, graceful shutdow
Executor (`executor.c`)	Complete	100%	fork+pipe, prime+matrix workloads
Protocol Layer (`protocol.c`)	Complete	100%	Fixed-size, byte-order safe
Unit Tests	Complete	100%	11 passing tests
Docker / Scripts	Complete	100%	Compose, scale, manage scripts
Dashboard (`dashboard.c`)	Stub only	0%	ncurses UI not yet implemented
Remote Code Execution	Not started	0%	Dynamic task dispatch from master
Raspberry Pi Deployment	Not started	0%	Real LAN demo (time-permitting)
Overall		**≈65%**	

8 Plan for Completing Remaining Work

8.1 Remote Code Execution on Workers (Core Priority)

Currently the executor only supports two hard-coded workloads (prime counting and matrix checksum). The primary Phase 3 goal is to extend the worker into a general-purpose **remote execution engine** that can receive and run arbitrary programs dispatched by the master.

The planned approach:

- **Extended protocol:** add a `MSG_EXEC` message type that carries a command string or binary payload in addition to the existing integer argument. The `NetworkPayload` struct will be extended (or a variable-length framing scheme added) to accommodate program names and arguments.
- **Executor extension:** `executor_run_task()` will use `execvp()` in the forked child to replace itself with the target program. The parent continues to collect output via the pipe and relay it to the master as a `MSG_RESULT`.
- **Security boundary:** only programs present in a whitelist directory on the worker will be executed, preventing arbitrary command injection from the master.
- **Output capture:** `stdout` and `stderr` of the child process will be redirected into the pipe so the master can log full program output, not just a single integer result.

This change makes the system a genuine distributed compute platform rather than a fixed-workload simulator.

8.2 Terminal Dashboard (Mohammed Yar)

The dashboard stubs in `src/ui/dashboard.c` need to be replaced with a full `ncurses` implementation. The planned layout uses three `WINDOW` panels:

- **Top panel** — live task queue depth (bar chart or counter).
- **Middle panel** — worker status table: ID, IP address, status, last heartbeat, current task.
- **Bottom panel** — scrolling log of recent events (task dispatch, results, disconnects).

A background `pthread` will copy a snapshot of the registry under lock every 500 ms and call `wrefresh()` to update the display without blocking the network threads.

8.3 Heartbeat Watchdog

The master currently records `last_heartbeat` timestamps but does not actively check them. A watchdog thread should mark workers `OFFLINE` if no heartbeat has arrived within a configurable timeout (e.g., 3× the heartbeat interval = 15 seconds), enabling automatic detection of silently crashed nodes.

8.4 Result Logging

Task completion events (worker ID, task ID, command, argument, result, wall-clock latency) should be written to a CSV log file for post-run performance analysis and graphing in the final report.

8.5 Additional Test Coverage

- `test_forking.c` and `test_protocol.c` exist in `tests/` but are not yet wired into the Makefile. These will be completed and integrated into `make test`.
- An end-to-end integration test script will verify task completion count and fault-tolerance behavior automatically without manual observation.
- Unit tests for the extended `execvp()`-based executor will be added once remote code execution is implemented.

8.6 Raspberry Pi Deployment (Time-Permitting)

If time permits, the system will be deployed on real hardware to demonstrate a genuine distributed LAN scenario:

- **Master node:** the development PC (x86-64, Linux/WSL) runs `bin/master`.
- **Worker nodes:** one or more Raspberry Pi devices (ARM, Linux) compile and run `bin/worker`, connecting to the PC's LAN IP address.
- **Cross-compilation:** the worker binary will be cross-compiled for ARM using `arm-linux-gnueabihf-gcc`, or compiled natively on each Pi.
- **Demo scenario:** the master dispatches a batch of compute tasks (prime counts, matrix checksums, or real programs via remote execution) across the Pi workers. The terminal dashboard on the PC shows live worker status, queue depth, and results arriving from physically separate machines.
- **Fault tolerance demo:** unplugging a Pi's network cable demonstrates automatic task requeue and system recovery in a real environment.

This deployment would serve as the capstone demonstration for the final report and validates that the system works correctly across heterogeneous architectures and a real network, not just localhost.

9 Repository Structure

```
distributed-resource-scheduler/
|-- src/
|   |-- master/
|   |   |-- queue_mgr.c / .h      # FIFO task queue
|   |   |-- scheduler.c / .h      # Round-robin scheduler + registry
|   |   +-- server.c / .h         # TCP server + main()
|   |-- worker/
|   |   |-- client.c / .h         # TCP client + heartbeat + main()
|   |   +-- executor.c / .h       # fork()+pipe() execution engine
|   |-- shared/
|   |   |-- models.h              # NetworkPayload, enums
|   |   +-- protocol.c / .h       # recv_full, send_full, byte-order
|   +-- ui/
|       +-- dashboard.c / .h      # ncurses stub (Phase 3)
|-- tests/
|   |-- test_scheduler.c          # 8 master unit tests
|   |-- test_worker_executor.c    # 3 worker unit tests
|   |-- test_forking.c            # (in progress)
|   +-- test_protocol.c           # (in progress)
|-- workloads/
|   |-- prime_calc.c              # Reference workload
|   +-- matrix_math.c             # Reference workload
|-- bin/                          # Compiled binaries
|-- Dockerfile
|-- docker-compose.yml
|-- Makefile
|-- start-workers.sh
|-- scale_workers.sh
|-- manage_workers.sh
+-- README.md
```